# Querying a Codebase with NVIDIA Auto Ontology

Will Cook

4 November 2026

*A source-to-SQL adapter, demonstrated on Auto Ontology's own repository*

## Abstract

Can a database agent answer questions about software if the source is represented as tables? We built **code-ontology-bridge**, a Python extractor that turns source files, symbols, calls and existing docstrings into relational data and a semantic model for NVIDIA Auto Ontology. Applied to Auto Ontology's own repository of 473 Python files and 5,319 symbols, we compared three ways of answering the same questions with the same language model: the schema alone, the schema plus the exported semantic model, and Auto Ontology with that model imported. On 16 held-out questions without hints, the semantic model raised NVIDIA Nemotron 3.5 Lightning from 6 to 13 correct answers and DeepSeek v4.1 Flash from 13 to 16. Inside Auto Ontology, DeepSeek answered all 22 development questions correctly, including a caller-chain query that depends on a function passed to `asyncio.to_thread`. The author wrote every question and reference query, about one repository, and each arm ran once, so these are measured differences on a small, single-repository test, not a general accuracy claim.

## 1. The question and the contribution

Suppose a developer changes `resolve_semantic_fks`, the function that resolves semantic foreign keys in Auto Ontology. Which callers reach it? Searching for its name finds an immediate caller. Reaching the scheduler also requires following a wrapper and recognizing a function passed as an argument. The question spans several files and two kinds of relationship: direct calls and callback references.

A source map can record those relationships once and make them available to SQL. NVIDIA Auto Ontology already has the other half of this workflow: an agent that retrieves semantic context, writes SQL and executes it against a connected database. This project connects the two. Instead of asking Auto Ontology about a sales database, we ask it about a database of its own implementation.

The work has three parts:

- **A source adapter.** A standalone extractor records Python source facts and exports DuckDB rows, model YAML and an evaluation dataset using Auto Ontology's existing contracts.
- **An exercised integration.** A compatibility patch configures public model clients and exposes an existing SQL routing option at the chat boundary. The retained run connects ingestion, model import, semantic compilation and question answering.
- **A controlled comparison.** Eleven complete runs over three arms, two models and two question sets, under a protocol committed before any run. Every request, SQL statement, result table and explanation is retained, including every miss.

The source-map approach comes from **CodeMap**, Will Cook's earlier source-navigation tool. This bridge adapts that approach into an installable package; it does not depend on CodeMap's private interface. NVIDIA supplies the database agent and semantic layer. The bridge supplies the source-derived data and its model.

## 2. How the source reaches the agent

Auto Ontology needs both a live database to query and semantic context that explains the database. The bridge exports them separately. Figure 1 follows the two paths to SQL execution.

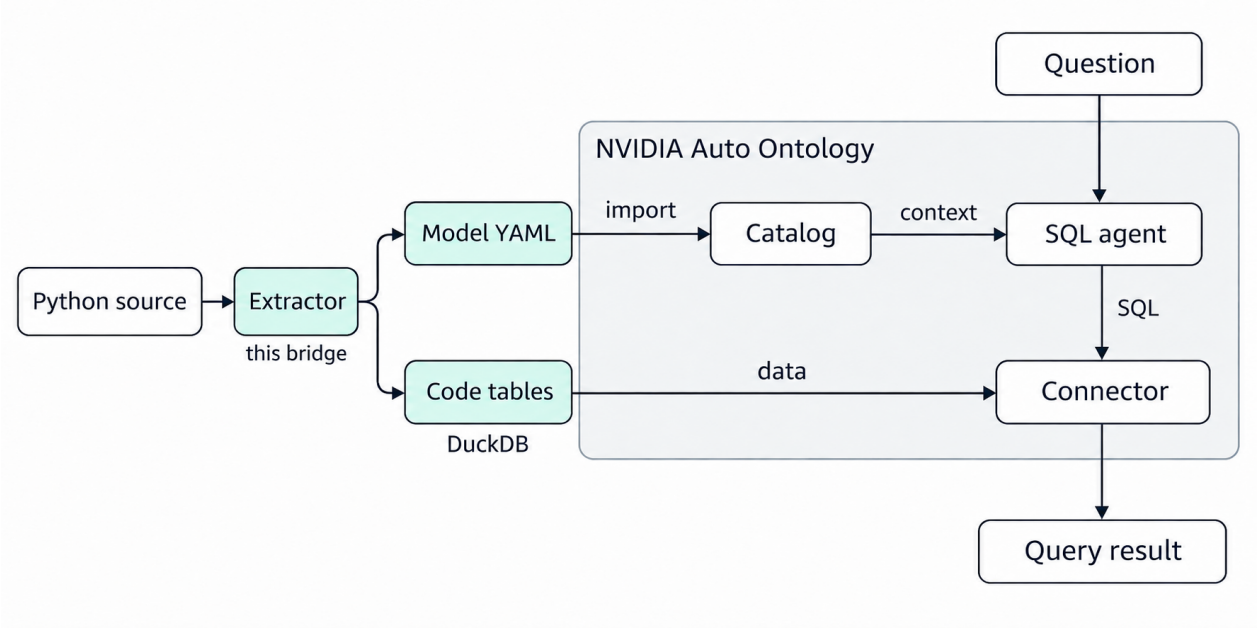



Figure 1: Two exports serve different purposes. DuckDB supplies source-code rows; model YAML supplies descriptions and relationships through import and compilation. The outlined components belong to Auto Ontology. The extractor is this bridge; the catalog is the semantic catalog and the connector is the existing DuckDB connector. This simplified query path omits ingestion services, model providers and answer narration.

The **DuckDB export** contains eight tables: files, docstring headings, symbols, class bases, calls, imports, test links and a source snapshot. Two views summarize incoming calls and public symbols without test associations. Auto Ontology reads these through its existing DuckDB connector.

The **model YAML** describes tables, terms, attributes and joins. It declares keys that the pinned DuckDB connector does not report. The live runner first ingests the database, maps the exported catalog identifiers to the server's identifiers, imports the model and compiles the semantic layer. Importing YAML alone does not create a queryable database.

A third export supplies DDL, CSV data and reference questions in NVIDIA's evaluation format. It provides a route toward a controlled comparison, but the full upstream benchmark harness was not run for this study.

### What is a row, and what is a definition?

The extractor uses Python's AST and preserves source paths and line ranges. A function becomes a row in `code.symbols`. Its docstring becomes a value in `summary`. A call becomes a row in `code.calls`, with identifiers for its caller and, when resolved, its callee. Joining symbols to files lets a query recover locations or exclude test files.

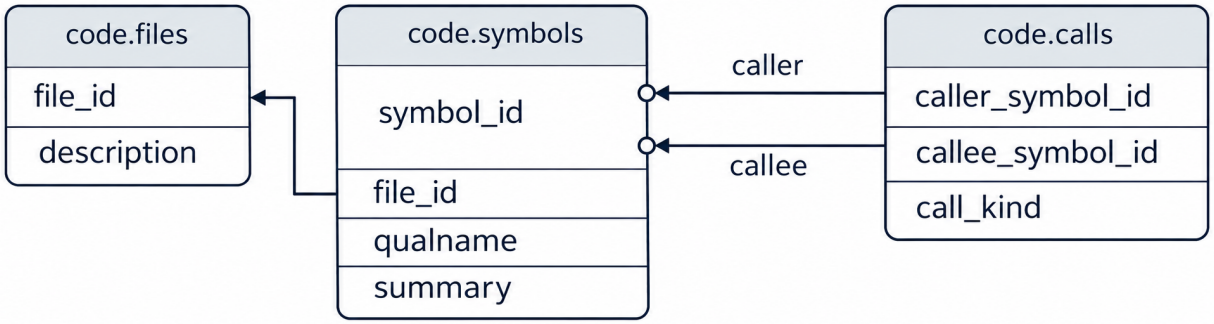



Figure 2: Selected columns show the joins used by a caller query. Each call references its caller and, when resolved, its callee; each symbol references its file. Descriptions of particular files and functions remain data values. The semantic model describes what the tables and columns mean.

This distinction keeps the model small: it defines a symbol as an entity type, rather than creating a separate semantic term for every function. Source descriptions are copied from existing docstrings. Missing descriptions stay empty; the extractor does not ask a model to invent them. The interface specification gives all columns, identity rules and resolution categories.

### One source occurrence, two representations

In `semantic_scheduler.py` under `auto_ontology/ingestion_service/`, line 203 of the pinned source passes the wrapper as an argument:

```
tables_processed = await asyncio.to_thread(
    run_semantic_compilation, database_name
)
```

The extracted row includes these fields:

```
call_kind: callback    resolution: imported
confidence: low       line: 203
```

The caller is `SemanticScheduler._run_once`, whose definition starts at line 132 in the scheduler file. The callee is `run_semantic_compilation`, defined in `compile.py` under `auto_ontology/semantic/`.

These selected fields from the frozen native model input show how Code symbol maps to the data:

```
id: cob:auto_ontology_code:term:symbols
name: Code symbol
represents:
  - cob:auto_ontology_code:table:code.symbols
columns_attributes:
  - id: cob:auto_ontology_code:attribute:code.symbols.symbol_id
    name: symbol id
    column_id: cob:auto_ontology_code:column:code.symbols.symbol_id
```

Below, IDs omit the shared `cob:auto_ontology_code:` prefix. The Call site term represents `table:code.calls`. Its semantic foreign keys connect two columns to the symbol-ID attribute:

- `column:code.calls.caller_symbol_id`
- `column:code.calls.callee_symbol_id`

Both target `attribute:code.symbols.symbol_id`. These definitions describe the joins; the particular functions and call occurrences remain database rows. The `summary` attribute's authored

description explains a source-docstring column, while each row's `summary` contains the original docstring. Both the queryable rows and the imported semantic context are needed. This is Auto Ontology's native format, not Ossie YAML.

The runner remaps exported catalog IDs before import. The frozen exported input and a later `/api/terms` response are retained; the submitted YAML, ID mapping and full compiled-model export are not. The terms response is a partial catalog readback and does not show which definitions a question retrieved. The inspection backend exposes these stages separately and supplies hash-checked source excerpts at build time.

### What changed upstream?

The data connector, model import and SQL pipeline already existed. The source extractor and exports need no changes to NVIDIA's code or model credentials. The recorded live run used Auto Ontology at `cdaf61e1bd0cf3820d11d72b43bb460816aebbd3`; compatibility with later upstream revisions has not been established. A separate twelve-file compatibility patch configures public model clients, forwards the existing `calculation_only` option through the chat API and extends a placeholder validator. It adds no code-specific connector, SQL engine or query planner.

Fixed SQL routing matters here because a question such as "What does this module do?" can sound like a metadata question. In this dataset, the answer is a value in a table. The patch selects the row-querying path for every question in the run. Its documentation and validation bind the patch to 611 passing tests in the upstream query, chat and utility scopes. That is scoped validation, not the full upstream test suite or evidence of upstream acceptance.

## 3. Following the caller chain

Question 11 asks which resolved caller symbols reach `resolve_semantic_fks`, directly or indirectly, excluding tests and reporting the shortest depth. This is a summary of the request: the full input also prescribed column names and types and instructed the agent to traverse resolved calls backwards, recursively, including callback references. Given those instructions, Auto Ontology generated recursive SQL over `code.calls` and returned four callers.

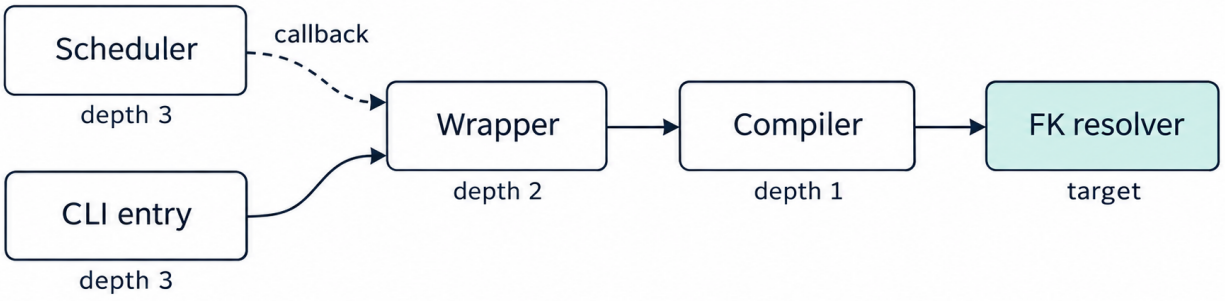



Figure 3: Four callers reach the foreign-key resolver through a wrapper and compiler. The table below identifies each source symbol. Arrows point from caller to callee, and SQL follows them in reverse. Solid edges are direct calls. The dashed edge records a callback passed to asyncio.to_thread, not proof that it ran.

| Diagram label | Source symbol | Depth |
|---|---|---|
| Compiler | `_run_semantic_compilation` | 1 |
| Wrapper | `run_semantic_compilation` | 2 |
| Scheduler | `SemanticScheduler._run_once` | 3 |
| CLI entry | `main` | 3 |

The target labeled "FK resolver" is `resolve_semantic_fks`; it is not a returned caller.

The locally computed inspection presets return three callers with direct calls alone and four when callback references are included; only the scheduler is added. This is a traversal of saved rows, not another model answer.

The scheduler passes `run_semantic_compilation` to `asyncio.to_thread`. The bridge records that argument as a callback reference, which connects the scheduler to the same chain as the ordinary caller `main`. The generated query follows reverse edges, avoids cycles and groups by full caller identity to select the minimum depth.

The saved response includes the request, intermediate events, final SQL and complete symbol identifiers. The request contains a hint about reverse traversal and callbacks; it is not an unaided discovery test. No saved SQL example was imported for this run. The answer establishes reachability within the extracted map. Dynamic calls can be missing, and a callback reference does not establish execution.

## 4. Evaluation: what each layer adds

The protocol, committed before any run, fixes three arms that use the same language model:

- **A, schema only.** The model sees table and column names and types, writes SQL, and may retry after an SQL error, up to three runs.
- **B, schema plus the semantic model.** The same, with each table's and column's description, the primary keys and the 10 declared joins from the exported model.
- **C, Auto Ontology.** Auto Ontology imports the model; its agent retrieves context, writes, checks and repairs the SQL, and runs it through the DuckDB connector.

Two models answered: NVIDIA Nemotron 3.5 Lightning through the NVIDIA API, and DeepSeek v4.1 Flash through OpenRouter. Arm C used `nvidia/nemotron-3-embed-1b` and `nvidia/llama-nemotron-rerank-vl-1b-v2` for retrieval. There are two question sets: 22 development questions written while the project was built, 18 with hints, and 16 held-out questions written for the protocol and committed before any arm ran, none with hints. Every request named its output columns; reference SQL and answer values were never sent. Each arm ran each set once, and no question was asked again.

The scorer requires named, typed columns, full case-sensitive identities, correct row multiplicity and requested ordering. Integers match exactly; other numbers allow absolute error up to 1e-6.

| Model | Arm | Development (22) | Held-out (16) |
|---|---|---|---|
| Nemotron 3.5 Lightning | A, schema only | 10 | 6 |
| | B, + semantic model | 13 | 13 |
| | C, Auto Ontology | 12 | no result |
| DeepSeek v4.1 Flash | A, schema only | 14 | 13 |
| | B, + semantic model | 18 | 16 |
| | C, Auto Ontology | 22 | 14 |

**The semantic model helps by itself.** B beats A for both models on both sets. On held-out questions Nemotron rises from 6 to 13 and DeepSeek from 13 to 16. Part of the reason is structural: at this revision the DuckDB connector reports no primary or foreign keys, so the model's declared keys and joins are the only place either agent can learn them.

**Auto Ontology adds more on development questions than on held-out ones.** Nemotron's one complete Auto Ontology run scored 12 of 22 development questions, against 13 with the metadata in the prompt; it ran through Auto Ontology's OpenAI-compatible client, which cannot switch off Nemotron's reasoning, so 1,120 of its structured-output calls used up the 8,192-token output limit and several answers came back empty. Its held-out run and three repeats through Auto Ontology's NVIDIA client with reasoning off were rate-limited by the NVIDIA API and are discarded, so Nemotron has no held-out result inside Auto Ontology. DeepSeek inside Auto Ontology answered all 22 development questions correctly, against 18 with the model in the prompt. On the held-out set it scored 14 against 16. One miss, Q101, follows an ambiguous wording: asked for the "functions or methods" a function calls, its SQL dropped a constructor call the reference counts. The other, Q106, admitted methods where the question asked for functions.

**Repairs matter.** In the DeepSeek development run, twelve of the 22 first queries named a table without its `code.` schema and failed with a catalog error. Auto Ontology's repair step fixed all twelve. Q2's rows were right, but its written explanation, which the agent writes with the SQL before the query runs, was the single word "Purple".

## 5. What the result establishes

The comparison shows that the exported semantic model makes two different models write correct SQL over a source-code database substantially more often, and that Auto Ontology's agent, given the model, can answer every development question correctly. It does not establish general accuracy. The author wrote all 38 questions and their references, about one repository; each arm ran once, and a difference of one or two of 16 questions is within what a second run could change. Matching rows on one database also does not prove query equivalence: an unnecessary filter can leave this snapshot's answer unchanged. Zhong, Yu and Klein (2020) evaluate SQL across test databases; this study's single-snapshot comparison has a narrower scope.

Before the comparison, two extractor defects were fixed and the database rebuilt: imports now bind only in the scope that contains them, and calls in a nested function count for its enclosing function only when that function calls or passes on the nested one. The rebuild removed 21 of 9,442 linked calls and added 7; the question 11 answer did not change.

The map itself is incomplete. Of 31,178 direct call sites, 9,442 resolve to an internal callee and 11,794 are classified as dynamic attribute calls without one. Another 1,046 module-level calls are omitted from the call table. The small residual category named `unresolved` does not measure all missing edges. Callback records have lower confidence; test links reflect imports or matching filenames rather than measured coverage. Repeated qualified definitions, such as property getters and setters, receive distinct line-qualified identities, while ambiguous targets remain unresolved.

Queryable source representations are established techniques. CodeQL builds databases for source analysis, and CodexGraph lets language-model agents query code graph databases. The contribution here is the adapter to Auto Ontology's contracts and its demonstrated use on source-derived rows.

## 6. Reviewing and extending the integration

For an Auto Ontology maintainer, the work separates into three reviewable pieces:

| Review question | Implementation or evidence |
|---|---|
| How do source facts enter the existing interfaces? | Extractor, exporters and interface specification. |
| What did the live service need? | Compatibility patch, with configuration, boundaries and scoped tests. |
| What did the agent actually produce? | Detailed results, linking every run, response and score. |

The immediate maintainer question is whether software repositories are a useful semantic-catalogue use case, and how public-model configuration and SQL-routing requirements should be supported.

The next tests are questions written by someone other than the author, a second repository, repeated runs to measure variation, and a comparison against an agent reading the source directly.

Separate governance commands record certification history, annotate SQL and reconcile definitions. They do not enforce certified-only retrieval or prove which definitions the agent used. This evaluation makes no new claim about those commands.

The runs used Auto Ontology `cdaf61e` and bridge `2d3686c`; the two DeepSeek Auto Ontology runs started at the next commit, `e1d41b3`, which changed only the website and the results summariser. Each Auto Ontology run's manifest binds full revisions, model settings, the database digest and frozen inputs. Follow the reproduction guide for extraction, compatibility tests or a new live run. Figure notes document the illustrations' sources and omissions.